Agente operador por pantalla para sistemas sin API

Documento de diseño · Reto 2 · Frontier CDMX 26 · Samuel Torres. Principio rector: la lógica determinista resuelve todo lo que puede; el modelo de lenguaje solo toma las decisiones que requieren interpretación.

1. Problema, alcance y supuestos

El sistema recibe una instrucción en lenguaje natural y las credenciales de un escritorio remoto, y debe completar la tarea en una aplicación que nunca ha visto: dejar exactamente los cambios esperados, responder una consulta o rechazar la tarea explicando la causa, en menos de cinco minutos y sin efectos colaterales.

- **Acceso:** la aplicación solo es observable como imagen (1280x800) a través de VNC; no hay DOM, API ni documentación.
- **Aislamiento:** cada tarea inicia en un escritorio limpio que puede pertenecer a otra aplicación; se procesa una tarea por escritorio a la vez.
- **Referencia temporal:** "hoy" corresponde a la fecha visible en la aplicación, no a la del servidor.
- **Fuera de alcance:** tareas que requieren información externa a la aplicación, captchas y operación concurrente sobre un mismo escritorio.

2. Riesgos identificados y mitigación

Riesgo	Mitigación
Repintado de pantalla de hasta 1 s	Captura repetida hasta obtener dos cuadros idénticos (máx. 1.5 s) antes de interpretar.
Clics menores a 100 ms ignorados / campos que concatenan	Pulsación sostenida de 150 ms; selección completa (Ctrl+A) antes de escribir.
Tareas imposibles o destructivas	Clasificación previa; se reporta *fallida* nombrando el dato que impide ejecutarla.
Registros duplicados y falso éxito	Búsqueda previa a toda creación; relectura del registro antes de reportar *terminada*.
Bucles de reintento del agente	Reglas en código: máximo dos reintentos por paso, bloqueo de acciones repetidas en la misma pantalla y corte a 4:30.
Cambios en la aplicación durante el día	Localización de elementos por texto, nunca por coordenadas; el mapa se invalida y re-explora si no coincide.
Exposición de secretos	Filtro que elimina contraseñas y tokens del resumen antes de enviarlo.

3. Arquitectura

- **Servicio HTTP (FastAPI):** autenticación Bearer; POST /tasks encola y responde el identificador en menos de 2 s; un worker ejecuta en segundo plano; GET /tasks/{id} expone estado y resumen.
- **Ciclo de control:** Clasificar, Percibir (captura + OCR local con coordenadas), Decidir (LLM de bajo costo vía OpenRouter, solo con texto), Actuar (cliente VNC), Verificar y Aprender. Se repite hasta cumplir el objetivo, determinar que es imposible o agotar el tiempo.
- **Grafo de navegación:** representación aprendida de la aplicación; cada pantalla es un nodo y cada acción una arista. Los recorridos exitosos se guardan como flujos reutilizables. La aplicación se identifica por su título en pantalla.
- **Persistencia:** JSON en memoria durante la tarea; MySQL en segundo plano con grafos, flujos, traza por paso, costos y errores. Despliegue en servidor Linux propio desde repositorio público.

4. Decisiones y alternativas descartadas

Decisión	Alternativa descartada y motivo
OCR local + LLM de texto	Enviar cada captura a un modelo de visión: costo elevado con US$45 de presupuesto y latencia incompatible con 5 min. La visión queda como respaldo.
El LLM elige la acción; el código la ejecuta	Que el LLM emita coordenadas: no es reproducible ni verificable.
Localización por texto	Coordenadas fijas: se rompen ante cualquier cambio de diseño.
JSON + MySQL asíncrono	Solo memoria: se pierde el histórico para depurar y reutilizar. Redis: sin concurrencia que lo justifique.
Exploración inicial de menús (~45 s)	Exploración a ciegas por tarea: más pasos y más llamadas al modelo. Se omite si la aplicación ya está registrada.
Implementación propia	Servicios de computer-use externos: prohibidos por las reglas y opacos para explicar fallas.

5. Plan de medición

- **Banco de pruebas:** las 5 tareas provistas, ejecutadas con /__reset, POST /tasks y /__check; se compara estado y respuesta contra lo esperado.
- **Metas:** al menos 4 de 5 correctas; 0 falsos éxitos; 0 acciones fuera de la tarea; menos de 3 min y US$0.20 por tarea; llamadas al modelo por tarea registradas.
- **Aprendizaje y robustez:** una segunda ejecución de la misma tarea debe requerir menos pasos; una tarea imposible inventada debe reportarse como fallida; un menú renombrado debe recuperarse por re-exploración.